

Shitcoin: A Peer-to-Peer Electronic Shitposting System

The Shitcoin Whitepaper - v1.0, September 2026

> *In the beginning there was Bitcoin. Then there were ten thousand copies of Bitcoin.

> This is the funniest one with the best tech.*

Abstract

Shitcoin (SHIT) is a fun-first cryptocurrency built on serious technology. It is a merge-mined, dual-layer blockchain that lets Bitcoin miners earn SHIT for free on the side, moves value between a UTXO payment layer and an EVM-compatible smart contract layer through a trustless bridge, and ships with native DeFi primitives: an automated market maker DEX (ShitSwap) and an on-chain lottery (ShitLottery) where every round burns 10% of the pot.

Shitcoin does not ask you to take it seriously. It asks you to take its engineering seriously while laughing. The codebase is a fork of battle-tested Bitcoin Core / Syscoin Core software: SHA-256 AuxPoW merged mining, LLMQ chainlocks, DIP-3 deterministic masternodes, and a full EVM. The meme is the marketing; the tech is the product.

Ticker: SHIT · Block time: 30 seconds · Consensus: PoW (merge-mined with Bitcoin) + masternode quorum services · Smart contracts: NEVM (EVM-equivalent, chain ID 57)

1. Introduction

Every cycle, the market re-learns the same lesson: the coins people love are the coins with personality, and the coins that survive are the coins with real infrastructure. Dogecoin proved the first half. Bitcoin proved the second.

Shitcoin is an attempt to have both at once - a coin you can laugh about that runs on infrastructure you don't have to laugh about.

Shitcoin's design goals, in order:

1. Be genuinely fun. The brand, the lottery, the community energy.

2. Be merge-mined with Bitcoin. Security should be inherited, not purchased.
3. Be useful. EVM smart contracts, a bridge, a DEX, and DeFi toys that actually work.
4. Be deflationary where it counts. Every lottery round permanently burns 10% of the pot.

Nothing in this paper is financial advice. SHIT is a meme coin with real tech, not real tech with a meme excuse.

2. Merged Mining with Bitcoin: Security for Free

2.1 The idea

Bitcoin miners perform an astronomical amount of SHA-256 hashing. Merged mining (AuxPoW, as pioneered by Namecoin and refined by Syscoin/Dogecoin) lets those same hashes simultaneously secure Shitcoin blocks at essentially zero marginal cost. A miner builds a Shitcoin block, embeds its hash in their Bitcoin block's coinbase, and if the Bitcoin block meets Shitcoin's (much lower) difficulty target, the Shitcoin block is valid.

Concretely, in this codebase:

`CAuxPow::check()` (`src/auxpow.cpp`) validates the parent Bitcoin block, the chain merkle branch, and the merged-mining header placement in the coinbase - the standard, audited AuxPoW construction.

Shitcoin's AuxPoW chain ID is 16 (testnet: 8), registered so parent coinbases can commit to it without colliding with other merge-mined chains.

Strict chain-ID enforcement (`fStrictChainId`) rejects parent blocks that claim our own chain ID, closing a class of spoofing attacks.

2.2 Why it matters

A new standalone PoW chain is born weak: low hashrate means cheap 51% attacks.

A merge-mined chain is born strong, renting Bitcoin's ~500 EH/s of security for the price of an extra merkle branch. Attacking Shitcoin costs as much as attacking Bitcoin itself, because the work is Bitcoin's work.

For miners, the pitch is simple: point your existing SHA-256 rigs at a Shitcoin-aware pool and collect SHIT block rewards on top of your BTC rewards.

No new hardware, no new electricity, no new heat. Free money for work you were already doing - the purest form of shitposting.

3. Dual-Layer Architecture: UTXO + NEVM

Shitcoin runs two execution environments in one client, because payments and programmability have different needs:

Layer 1 - UTXO (the Bitcoin layer). Fast, simple, auditable payments.

30-second blocks, native assets, aliases, and instant probabilistic confirmations (ZDAG-style). This is where SHIT lives as money.

Layer 2 - NEVM (the Ethereum layer). A full EVM-equivalent runtime (chain ID 57 on mainnet) embedded in the node. Any Solidity contract, any Ethereum wallet, any EVM tooling - it all just works, except gas is paid in SHIT instead of ETH.

The two layers share one validator set (the masternode quorum network) and one security budget (Bitcoin-merged mining). Users move SHIT between layers through the bridge (Section 4) without trusting any custodian.

4. Bridge Tech: The Trustless Two-Way Peg

"Bridge tech, if possible" - good news: it's already in the codebase, and this section specifies how Shitcoin uses it.

4.1 UTXO <-> NEVM bridge (native)

Moving SHIT between the payment layer and the contract layer works like this:

UTXO -> NEVM (mint): the user burns SHIT in a special UTXO transaction.

Relayers submit an SPV proof of that burn to the Vault Manager contract

(0x7904299b3D3dC1b03d1DdEb45E9fDF3576aCBd5f on mainnet parameters). Once the proof is verified against enough PoW, the contract mints the same amount of SHIT on NEVM. No custodian ever holds the funds; the burn is the deposit.

NEVM -> UTXO (burn/withdraw): the user destroys NEVM SHIT in the Vault Manager. The masternode quorum observes the event, co-signs a UTXO release, and the funds reappear on the payment layer.

Trust model: SPV proofs + a deterministic masternode quorum (DIP-3), not a multisig committee pinky-swearing. The bridge is as decentralized as the masternode set, and every step is verifiable on-chain.

4.2 Bridging outward

The same construction generalizes: any chain that can verify SPV proofs (or run a light client of Shitcoin) can peg SHIT in and out. The reference design follows the sysethereum pattern - a relay contract on the foreign chain plus the masternode quorum as decentralized relayers. New bridges are governance proposals (Section 8), not hard forks.

5. EVM Compatibility and Codebase Updates

The NEVM is a fork of go-ethereum maintained alongside this repository. The standing policy for "updating the EVM codebase":

1. Track upstream geth. Rebase the fork onto each stable go-ethereum release; the only carried patches are the Shitcoin precompiles (bridge proof verification, quorum randomness hooks) and the SHIT-as-gas changes.
2. Keep the precompile surface minimal. Every custom precompile is consensus-critical; additions require an audit note in doc/.
3. Solidity-first development. All Shitcoin DeFi (ShitSwap, ShitLottery) ships as ordinary Solidity targeting the NEVM - no custom tooling, so Ethereum developers are productive on day one.

Because the EVM is equivalent (not merely "compatible"), contracts deployed on Ethereum can be redeployed on Shitcoin's NEVM unchanged - bringing the entire Ethereum DeFi toolbox to a merge-mined chain with 30-second finality vibes and joke-tier branding.

6. ShitSwap: The Native DEX

A fun coin needs a casino floor. ShitSwap is Shitcoin's reference automated market maker, deployed on NEVM and fully compatible with this codebase's EVM.

6.1 Design

Constant-product AMM ($x * y = k$), the battle-tested Uniswap V2 design: permissionless pair creation, LP tokens, 0.3% swap fee accruing to liquidity providers.

Factory + pair architecture (contracts/ShitSwap.sol): one factory deploys minimal pairs; each pair is its own ERC-20 LP token.

SHIT as the hub asset. Every serious pair routes through SHIT, so all liquidity ultimately deepens the SHIT market. WSHIT (wrapped/bridged SHIT) pairs let UTXO-native assets trade against EVM tokens once bridged.

No admin keys in the core. Fees and pair creation are governed by the masternode proposal system, not a dev multisig.

6.2 Why an AMM fits Shitcoin

Order books need market makers; meme coins have gamblers. An AMM turns every holder into a passive market maker and every trade into exit liquidity with better vibes. Combined with the bridge, ShitSwap lets someone swap a UTXO asset for an NEVM meme token in two transactions - the kind of UX that makes a fun chain actually get used.

7. ShitLottery: Provably-Fair Degeneracy

The centerpiece toy: an on-chain lottery where **players pay SHIT for tickets, one ticket wins the whole pot at random, and 10% of every pot is burned forever.**

7.1 Rules

1. Rounds. The lottery runs in consecutive rounds. Each round has a fixed ticket price (e.g. 100 SHIT) and a ticket-buying window measured in blocks.
2. Tickets. Anyone calls `buyTickets(n)` and sends $n \times \text{price}$. Each ticket is one entry; buying more tickets means more chances. All funds sit in the contract - no custody, no operator.
3. The draw. After the window closes, anyone may call `drawWinner()`. The winner index is derived from the block hash of the closing block mixed with the round ID: `winner = uint(keccak256(blockhash(closeBlock), roundId)) % ticketCount``. The draw is permissionless - if nobody calls it, the pot just

waits.

4. The split. Winner receives 90% of the pot. The remaining 10% is sent to the burn address (0x00000000000000000000000000000000dEaD), permanently removing it from supply. Every round is a small deflationary event.
5. Next round. A new round opens automatically; unclaimed edge cases (a round with zero tickets) simply roll over.

Reference implementation: `contracts/ShitLottery.sol`.

7.2 On randomness, honestly

Block-hash randomness is manipulable by whoever produces the closing block - they can discard a block that makes them lose (at the cost of the block reward). For a fun, low-stakes lottery this is an acceptable and fully disclosed trade-off, and the permissionless draw means no operator can stall or rig the timing. The whitepaper-recommended production upgrade path is a masternode-quorum randomness beacon (the LLMQ network already produces threshold signatures - the natural decentralized randomness source on this chain) or an external VRF oracle. Ship the fun version first; harden the randomness as the pots grow.

7.3 Why burn 10%?

Two reasons. Economically, every lottery round is a buy-and-burn engine that counteracts tail emission - the more fun people have, the scarcer SHIT gets. Psychologically, watching the burn counter climb is half the entertainment. The whitepaper's position: a meme coin should have at least one mechanism that is unironically good tokenomics, and this is it.

7.4 Native Vault: lock it and forget it

Not everyone wants to gamble. The native vault lets holders lock SHIT for 1 to 5 years and earn yield while they wait - enforced by consensus, not by any custodian:

Lock duration	APY
1 year	4%

Lock duration	APY
2 years	6%
3 years	9%
4 years	12%
5 years	15%

How it works: vaultlock creates a P2WSH output whose witness script is

<locktime> CHECKLOCKTIMEVERIFY DROP <your-pubkey> CHECKSIG - a strict

timelock only your key can ever spend, and only after maturity. There is **no**

early exit^{**}: lock it and forget it, literally. When you vaultclaim after

maturity, the network itself mints your yield -

principal x (tierMultiplier ? 1), compounded per the table above - and pays

it alongside your principal. No human approves, funds, or can interrupt

anything; it works like a block reward.

Trustless funding. Vault yield is paid from the 30% vault yield reserve

(6,300,000,000 SHIT). This reserve is not coins held by anyone - it was

never sent to an address. It is a protocol-level number every node tracks,

like the block subsidy schedule, and new SHIT for yield is minted directly

from it. There is no private key because there is no address; not even the

team can touch it. Two consensus caps bound it: at most **84M SHIT** of yield

per year^{**}, and 6.3B SHIT total - so the reserve is mathematically guaranteed

to last the full 75-year schedule. If it is ever exhausted, locks still

return principal in full; they simply earn no further yield.

Bitcoin holders welcome. To lock or claim, the official wallet requires

you to hold any amount of bitcoin (even a single satoshi) in a BTC

address you prove you own by signing a challenge message. This is enforced by

the wallet software as a matter of policy - the Shitcoin chain cannot observe

the Bitcoin chain - and it keeps the vault aligned with the Bitcoin community

the coin is merge-mined alongside.

The EVM ShitVault contract (contracts/ShitVault.sol) remains as a DeFi

companion with its own 10%-burn early-exit mechanic, but the canonical,

trustless reserve described above lives at consensus level.

8. Tokenomics

Max supply: 21,000,000,000 SHIT (21 billion). No tail emission - the last new SHIT is minted roughly 50 years after genesis. Allocation:

Bucket	Share	Amount	Notes
Team	18%	3,780,000,000	Core team & founders
Coin support	2%	420,000,000	10% of the 20% team bucket: listings, liquidity, marketing, ops
Vault yield reserve	30%	6,300,000,000	Protocol-level reserve (no custodian); minted as vault yield, capped at 84M SHIT/year over 75 years
Mining	50%	10,500,000,000	Block rewards (see below)

The team and coin-support allocations are paid in block 1 - the coinbase must contain exactly two outputs matching those amounts, enforced in consensus (`CheckAllocationBlock()` in `src/validation.cpp`), so the schedule cannot be altered after launch. The 30% vault reserve is not paid in block 1: it exists only as a consensus-tracked reserve that can solely be minted as vault yield (`CheckVaultYield()`), capped per year and in total. The genesis block's 50 SHIT is unspendable, as is tradition.

Emission. Mining starts at block 2 at ~2,705.31 SHIT/block, declining 5% per year over a 50-year tail. The base subsidy is tuned so lifetime mining sums to exactly the 10.5B allocation.

Reward split (per block): 10% to the governance superblock -> of the remaining 90%: 25% to miners (merge-mined, Section 2), 75% to masternodes. 50% of transaction fees also go to masternodes.

Masternodes. 100,000 SHIT collateral, DIP-3 deterministic registration, providing quorum services: ChainLocks (instant finality), bridge relaying, and governance voting.

Governance. Monthly superblocks fund proposals voted on by masternodes - bridge deployments, EVM updates, lottery parameter tweaks, and marketing.

Vault yield needs no governance: it is minted automatically by consensus within the 75-year / 84M-per-year caps.

Burns. The lottery's 10%-per-round burn and the EVM vault's 10%

early-exit penalty are the protocol's deflationary sinks; both only reduce circulating supply below the 21B cap.

9. Network & Consensus

30-second blocks, SHA-256 PoW, AuxPoW merged mining with Bitcoin.

LLMQ ChainLocks (llmq400_60): masternode quorums sign each block, giving near-instant finality and 51%-attack immunity on top of merged mining.

DIP-3 deterministic masternodes, DIP-19 data handling, full Taproot/Segwit support inherited from the Bitcoin Core base.

Network identity: mainnet P2P magic ce e2 ca ff, default port 8369, bech32 HRP sys-family (chain params in src/kernel/chainparams.cpp).

10. Roadmap (subject to vibes)

Phase 1 - Launch the joke properly. Rebrand binaries, public testnet, mining pools add SHIT AuxPoW, first ShitLottery round on NEVM testnet.

Phase 2 - DeFi toys. ShitSwap mainnet deployment, lottery randomness beacon via LLMQ threshold signatures, bridge UI.

Phase 3 - Expansion. Outward bridges (per Section 4.2), CEX listings driven by community governance proposals, merch (you know you want the shirt).

Phase 4 - World domination. Unlikely. But funny to write down.

11. Risks & Disclaimers

SHIT is a meme coin. Buy it for fun, not as an investment strategy. Nothing in this paper is financial advice.

Smart contracts (ShitSwap, ShitLottery, bridge contracts) carry bug risk; the reference implementations are starting points, not audits.

Bridge and lottery trust assumptions are documented above - read them before bridging life savings (don't bridge life savings).

Regulatory treatment of meme coins, lotteries, and DeFi varies by jurisdiction; the lottery's legality where you live is your homework.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
2. Namecoin / Syscoin AuxPoW merged-mining construction (src/auxpow.cpp).
3. Dash DIP-0003 (deterministic masternodes), DIP-0019.
4. Uniswap V2 core (constant-product AMM design), 2020.
5. Syscoin SYSX / sysethereum bridge design docs.
6. This repository: src/kernel/chainparams.cpp (network & emission parameters),
src/validation.cpp (GetBlockSubsidy), contracts/ (ShitSwap, ShitLottery,
ShitVault - with compiler build artifacts in contracts/build/).

*© 2026 The Shitcoin Developers. Released under the MIT License, like the
codebase it describes. Don't be dumb with your money.*